

18. CallableStatement (or) Stored Procedure

A CallableStatement is a Java interface in the JDBC API that allows a Java application to call stored procedures in a database. Stored procedures are precompiled SQL code stored in the database, which can accept parameters and perform a wide range of database operations such as inserting, updating, deleting, or returning results.

CallableStatement interface is used to call the stored procedures and functions. We can have business logic on the database by the use of stored procedures and functions that will make the performance better because these are precompiled. Suppose you need to get the age of the employee based on the date of birth, you may create a function that receives date as the input and returns age of the employee as the output.

18.1 What is the difference between stored procedures and functions

The differences between stored procedures and functions are given below:

Stored Procedure	Function
is used to perform business logic.	is used to perform calculation.
must not have the return type.	must have the return type.
may return 0 or more values.	may return only one values.
We can call functions from the procedure.	Procedure cannot be called from function.
Procedure supports input and output parameters.	Function supports only input parameter.
Exception handling using try/catch block can be used in stored procedures.	Exception handling using try/catch can't be used in user defined functions.

What is a Stored Procedure?

A stored procedure is a set of SQL statements that are precompiled and stored in the database. They are reusable and allow for modular programming within the database. Stored procedures can:

- Accept input parameters.
- Return output parameters.
- Perform complex operations, including loops and conditional logic.
- Improve performance by reducing the need for multiple SQL statements sent from the application.

Example to call the stored procedure using JDBC

To call the stored procedure, you need to create it in the database. Here, we are assuming that stored procedure looks like this.

```
DELIMITER $$
```

```
DROP PROCEDURE IF EXISTS `organization`.`InsertCompanyRecord` $$
CREATE PROCEDURE `organization`.`InsertCompanyRecord` (
    IN p_name VARCHAR(50),
    IN p_department VARCHAR(50),
    IN p_salary DECIMAL(10, 2)
)
BEGIN
    INSERT INTO company (name, department, salary)
    VALUES (p_name, p_department, p_salary);
END$$
```

```
DELIMITER ;
```

The table structure is given below:

```
use organization;
CREATE TABLE company (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50),
    department VARCHAR(50),
    salary DECIMAL(10, 2)
);
```

```
package com.lokesh;

import java.sql.*;

public class CallStoredProcedure {

    public static void main(String[] args) {

        String jdbcURL = "jdbc:mysql://localhost:3306/organization";
        String username = "root";
        String password = "root";

        // Values to be inserted
        String name = "Alice Johnson";
        String department = "Finance";
        double salary = 68000.00;

        // SQL to call the stored procedure
        String sql = "{CALL InsertCompanyRecord(?, ?, ?)}";

        Connection connection = null;
        CallableStatement callableStatement = null;
```

```

try {
    // Establish connection
    connection = DriverManager.getConnection(jdbcURL, username, password);

    // Prepare the callable statement
    callableStatement = connection.prepareCall(sql);

    // Set input parameters
    callableStatement.setString(1, name);
    callableStatement.setString(2, department);
    callableStatement.setDouble(3, salary);

    // Execute the stored procedure
    callableStatement.execute();
    System.out.println("Record inserted successfully.");

} catch (SQLException e) {
    e.printStackTrace();
} finally {
    // Close resources explicitly
    try {
        if (callableStatement != null) {
            callableStatement.close();
        }
        if (connection != null) {
            connection.close();
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
}

```

Output:

```
Record inserted successfully.
```

Example to call the function using JDBC

Let's create the simple function in the database first. If you cannot or do not want to modify the function, you can adjust the `log_bin_trust_function_creators` system variable to bypass this restriction. However, this is less secure and not recommended in production environments.

1. Check the current setting of `log_bin_trust_function_creators`:
`SHOW VARIABLES LIKE 'log_bin_trust_function_creators';`
2. Set `log_bin_trust_function_creators` to 1 (requires superuser privileges):
`SET GLOBAL log_bin_trust_function_creators = 1;`

3. Create your function:
`DELIMITER $$`

```

CREATE FUNCTION `organization`.`AddTwoNumbers` (
    num1 DOUBLE,
    num2 DOUBLE

```

```

) RETURNS DOUBLE
BEGIN
    RETURN num1 + num2;
END$$

```

```

DELIMITER ;

```

```

package com.lokesh;

import java.sql.*;

public class CallMySQLFunction {
    public static void main(String[] args) {

        String jdbcURL = "jdbc:mysql://localhost:3306/organization";
        String username = "root";
        String password = "root";

        // Values to pass to the function
        double num1 = 10.5;
        double num2 = 20.3;

        // SQL to call the function
        String sql = "SELECT AddTwoNumbers(?, ?) AS Result";

        Connection connection = null;
        PreparedStatement preparedStatement = null;
        ResultSet resultSet = null;

        try {
            // Establish connection
            connection = DriverManager.getConnection(jdbcURL, username, password);

            // Prepare the statement
            preparedStatement = connection.prepareStatement(sql);
            preparedStatement.setDouble(1, num1);
            preparedStatement.setDouble(2, num2);

            // Execute the query
            resultSet = preparedStatement.executeQuery();

            // Process the result
            if (resultSet.next()) {
                double result = resultSet.getDouble("Result");
                System.out.println("The sum of " + num1 + " and " + num2 + " is: " + result);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            // Close resources
            try {
                if (resultSet != null) resultSet.close();
                if (preparedStatement != null) preparedStatement.close();
                if (connection != null) connection.close();
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
}

```

Output:

```
The sum of 10.5 and 20.3 is: 30.8
```